

ML-Based Flood Prediction System Using Historical Threat Mapping and Live Weather APIs

Submitted to: Dr. Jeelani

Amit Bishnoi (1024240005)

Diksha Garg (1024240015)

Thapar Institute of Engineering and Technology

August 2026

1 Introduction & Problem Statement

Floods cause massive damage around the world. Because of climate change, they are happening more often, making it crucial to have better ways to predict them. Right now, weather apps show us live data like rain and wind, but they do not tell us if that rain will actually cause a flood in a specific area.

On the other hand, traditional flood maps are usually drawn once and rarely updated. They cannot react to a sudden, heavy storm. Also, databases that track past disasters (like EM-DAT) have great details on the damage caused, but they often lack exact GPS coordinates. Because of this, current systems struggle to instantly connect a live storm with a location's history of flooding.

Our project aims to fix this gap. We are building a system that takes live weather data and compares it against a combined database of past floods. By merging the exact GPS data from the Dartmouth Flood Observatory (DFO) with the impact records from EM-DAT, our machine learning model will be able to predict the real-time flood risk for any searched location.

2 Literature Review

Past flood prediction relied heavily on complex physical models of rivers and terrain, which require immense computing power and are difficult to scale for smaller cities. While Machine Learning (ML) has emerged as a faster alternative to predict water levels, current systems largely fail to integrate live weather API data with long-term disaster records. This project bridges that gap by treating geographic vulnerability as dynamic, mixing real-time weather conditions with historical data to calculate an instant, automated risk score.

3 Objectives

The main goal of this project is to build a machine learning system that uses live weather data and past disaster records to predict current flood risks.

Specific steps to achieve this include:

- To pull real-time weather data (like rainfall, temperature, and wind speed) using the OpenWeatherMap API.
- To clean and combine the EM-DAT and DFO datasets to create a complete history of floods with exact GPS coordinates.
- To create custom data features for our model, such as past flood severity and how much it rained in the last 48 hours.
- To train and test machine learning models to predict if the flood risk is Low, Medium, or High.
- To build a simple user interface where anyone can search a location and see its live weather and risk score.

4 Methodology & System Architecture

The project relies on a five-stage data pipeline, taking raw live signals and turning them into a final risk score. The overall system architecture is illustrated in Figure 1.

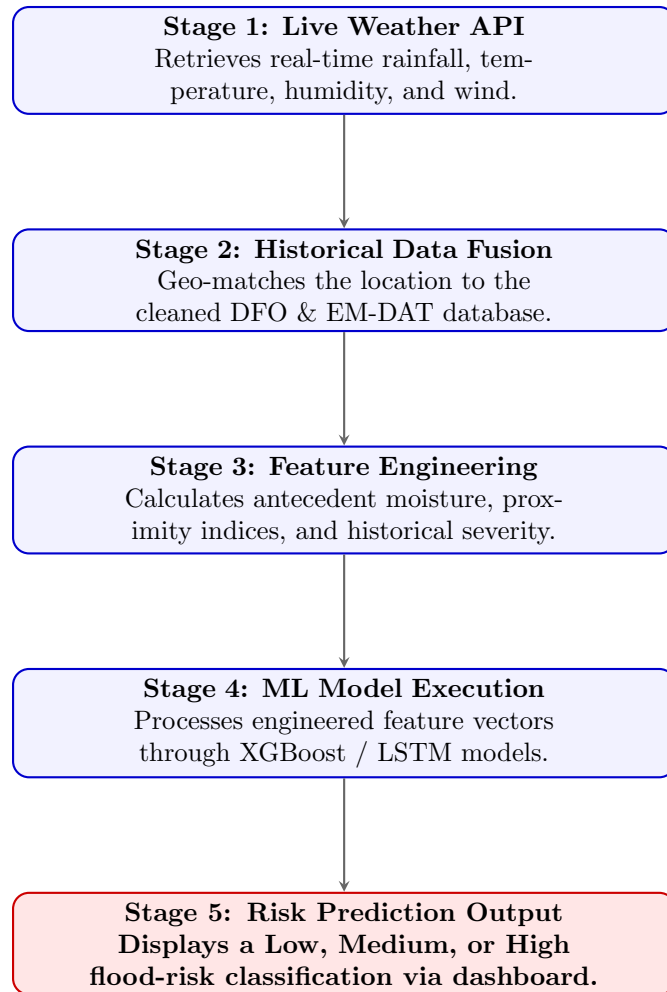


Figure 1: System Architecture: End-to-end data pipeline from live signal to classified risk score.

4.1 Stage 1: Live Weather API

The system starts by searching a specific location. It pulls real-time rainfall, temperature, humidity, and wind speed for that exact spot using the OpenWeatherMap REST API.

4.2 Stage 2: Historical Data Fusion

The historical data pipeline addresses the spatial limitation of EM-DAT by merging it with DFO centroids, as illustrated in Figure 2. We clean and filter historical flood events using our fused database, geo-matching past events to the user’s searched coordinates.

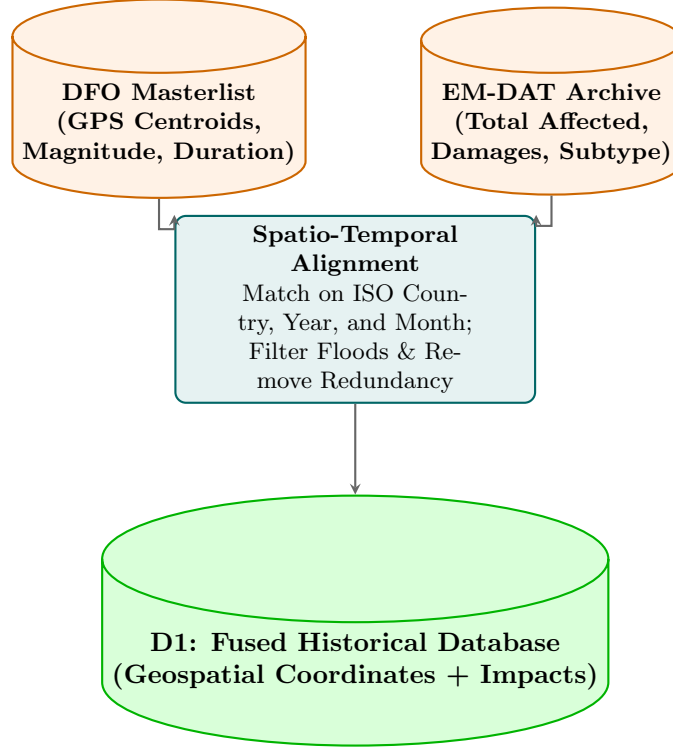


Figure 2: Dual-Dataset Fusion Workflow: Integrating spatial resolution with impact severity.

4.3 Stage 3: Feature Engineering

In this stage, raw parameters are transformed into normalized numerical vectors. This includes calculating the spatial proximity index (using the Haversine distance to historical flood centroids), antecedent moisture metrics, and multi-hour cumulative precipitation.

4.4 Stage 4: ML Model Training

The processed vectors are fed into machine learning classifiers to predict risk categories:

- **XGBoost / Random Forest:** Best suited for tabular tabular feature vectors, yielding calibrated multi-class probabilities (Low, Medium, High).
- **LSTM Networks:** Evaluated for multi-step temporal dependencies where continuous sequential rainfall forecasts are available.

4.5 Stage 5: Risk Prediction Output

The platform aggregates the probability scores, establishes confidence intervals, and renders a localized threat level alongside live weather cards on an interactive Streamlit dashboard.

5 Evaluation Criteria & Validation Framework

To ensure clinical rigor and eliminate data leakage, the model is evaluated on a partitioned 20% holdout test set. Figure 3 outlines the comprehensive evaluation and metric validation protocol.

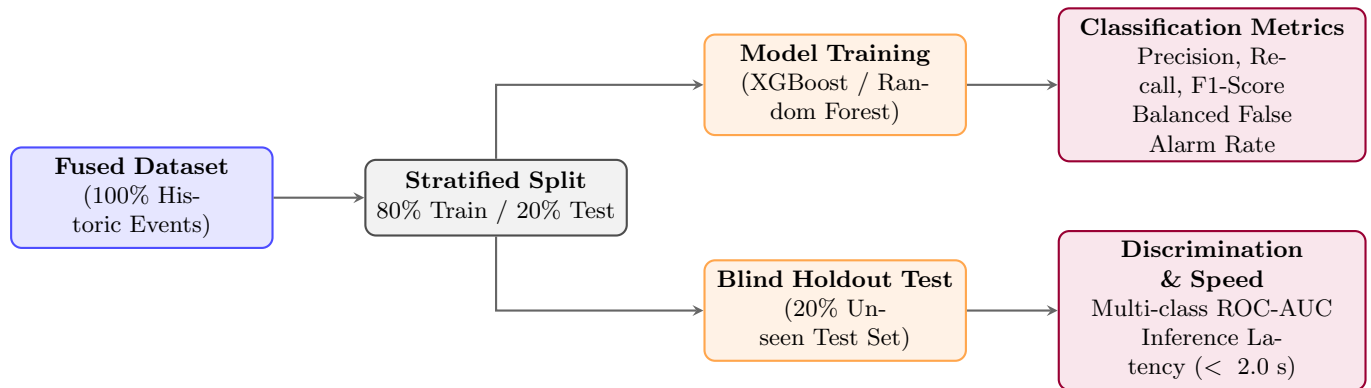


Figure 3: Model Evaluation Protocol: Stratified holdout validation, multi-class assessment, and system latency benchmarking.

The specific performance metrics include:

- **Precision, Recall, and F1-Score:** Critical for balancing false alarms (false positives) against missed emergency warnings (false negatives).
- **ROC-AUC:** Assesses the model’s ability to discriminate between Low, Medium, and High threat thresholds across various classification cutoffs.
- **System Latency:** Benchmarking inference and feature engineering time to ensure the complete dashboard response is generated in under 2 seconds.

6 Project Timeline

Phase	Task Description	Key Deliverable
Weeks 1–3	Read existing research, set up API keys, and clean the data.	Cleaned dataset & working API code
Weeks 4–7	Combine DFO and EM-DAT data, and start training ML models.	Trained models & accuracy reports
Weeks 8–10	Connect the backend code to the front-end user interface.	Working software prototype
Weeks 11–12	Fix bugs, improve the design, and write the final report.	Final project file & presentation

7 Resources & Risk Assessment

7.1 Resources Needed

We will use free, open-source Python libraries (like Scikit-Learn and Pandas) and public datasets (EM-DAT and DFO). The live weather data will come from the free tier of the OpenWeatherMap API. Computation will be executed on local machines with lightweight footprints, resulting in zero external budget requirements.

7.2 Possible Risks & Solutions

- **Mismatched Data:** When combining EM-DAT and DFO, timestamps or regional boundaries may have variance. *Solution:* We employ fuzzy temporal matching (matching events within the same month/year and country) and spatial nearest-neighbor verification.
- **Imbalanced Data:** The dataset naturally contains fewer severe flood events than normal atmospheric days. *Solution:* We utilize class weighting and Synthetic Minority Over-sampling Technique (SMOTE) to prevent majority-class bias.
- **API Rate Limits:** Free weather APIs enforce request frequency caps. *Solution:* A local response caching store (D2) retains recent queries to avoid redundant API pings during active testing.

8 Summary

This project proposes a deployable, machine learning-driven platform to reduce flood-related damages by providing instant, localized risk assessments. By successfully fusing high-resolution spatial data with historical impact metrics and live weather APIs, it overcomes the severe limitations of traditional, static flood maps. The system is designed to be accessible, cost-effective, and highly scalable, offering a vital early warning tool for both everyday users and emergency responders.